

COP4620 Compiler Construction

Practice Problem

myFilter — a higher-order function in Java, Python, and Haskell

Practice for the myMap Signature Assignment. Where `map` transforms every element, `filter` keeps a subset — so the result can be shorter than the input.

1. The Task

Write a higher-order function named `myFilter` in three languages — Java, Python, and Haskell. `myFilter` is passed two things: a **list of integers** and a **predicate** (a function that takes an integer and returns true or false). It returns a **new list** containing only the elements for which the predicate returns true, in their original order. The original list must not be modified.

2. What myFilter Must Do

- Inputs: a list of integers and a predicate `p : Int -> Bool`.
- Output: a new list of exactly the elements `x` of the input for which `p(x)` is true.
- The relative order of the kept elements is preserved.
- The output length varies with the predicate — anywhere from 0 up to the full input length.
- **No side effects:** never mutate the input list; build and return a new one.
- The predicate is a parameter — the same `myFilter` must work for ANY `Int→Bool` predicate the caller supplies.

3. Examples

Predicate	Input	Result
keep evens <code>x % 2 == 0</code>	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]	[2, 4, 6, 8, 10]
keep <code>x > 5</code>	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]	[6, 7, 8, 9, 10]
keep negatives <code>x < 0</code>	[3, -1, 4, -1, 5]	[-1, -1]
keep all (always true)	[7, 7, 7]	[7, 7, 7]
keep none (always false)	[7, 7, 7]	[]

4. Suggested Signatures

You may use these signatures (they match the style your instructor expects):

Language	Signature
Java	<code>List<Integer> myFilter(List<Integer> xs, IntPredicate keep)</code>

Python	<code>def my_filter(xs: list[int], pred: Callable[[int], bool]) -> list[int]</code>
Haskell	<code>myFilter :: (Int -> Bool) -> [Int] -> [Int]</code>

5. What Each Version Should Show

- **Java:** accept the predicate as a functional interface (`IntPredicate`) and call `keep.test(x)`; build and return a new `ArrayList`; clean iteration.
- **Python:** write the logic yourself (a loop or a list comprehension) — do *not* simply call the built-in `filter()`; return a new list.
- **Haskell:** give the correct type signature and use recursion + pattern matching with guards (or a `foldr`); keep it pure.
- **All three:** run the same test case in each language and show the outputs match, and briefly explain how each version reflects its paradigm (Java: imperative + functional interface; Python: interpreted scripting + first-class functions; Haskell: pure functional recursion/fold). Include build/run instructions.

6. Expected Output (self-check)

A correct solution that filters `[1..10]` by `x % 2 == 0` and then by `x > 5` should print exactly these lines in all three languages:

```
input : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens : [2, 4, 6, 8, 10]
gt 5   : [6, 7, 8, 9, 10]
original after calls (unchanged): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The last line reprints the original list to confirm `myFilter` did not modify its input.

7. What to Submit & How to Run

Submit: `MyFilter.java`, `my_filter.py`, and `MyFilter.hs` (plus a short note on the paradigms if your instructor asks).

Java

```
javac MyFilter.java
java MyFilter
```

Python

```
python3 my_filter.py
```

Haskell

```
runghc MyFilter.hs
# or build a binary:
ghc -o myfilter MyFilter.hs && ./myfilter
```

8. Tips & Going Further

Tips:

- Filter is different from map: you decide whether to *keep* each element, so the result length can change.

- Test the edge cases: an empty list, a predicate true for everything, and a predicate true for nothing.
- Build a new list — never modify the one you were given.

Going further (optional):

1. Combine with map: square only the even numbers (filter, then map).
2. Write `myReject`, the opposite of `filter`, by negating the predicate.
3. Generalize from `Int` to any type: Java generics `<T>`, Python `TypeVar`, Haskell `(a -> Bool) -> [a] -> [a]`.